

Reviewer Pack — Minimal Hodge Theoretical Dimension and Communication Comple...

Entry af50c556b5f0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 21:27:45 UTC

Minimal Hodge Theoretical Dimension and Communication Complexity Rank

Entry ID: af50c556b5f0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 21:27:45 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every n -vertex graph G , the minimal Hodge theoretical dimension ($\text{hdim}(G)$) of its associated matroid is linearly correlated with its communication complexity rank ($\text{ccr}(G)$), such that $\text{hdim}(G) = \Theta(\text{ccr}(G))$.

Rationale (proposer's reasoning):

Hodge theory, which studies the geometry of algebraic varieties, provides a rich framework for understanding the topological properties of spaces. Communication complexity, which measures the amount of information required to transmit data between two parties, has been related to the rank of the associated matroid. This conjecture aims to explore the relationship between these two fields by proposing that the Hodge theoretical dimension of a graph's matroid is proportional to its communication complexity rank.

Taxonomy category: Geometric-Invariants-CommunicationComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 07c63b73bc661bf7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $\text{hdim}(G)$ and $\text{ccr}(G)$ for all n -vertex graphs ($n \leq 40$) exceeds 0.8, and the p-value from the correlation test is less than 0.05.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge theory" AND "communication complexity" AND "matrix rank" - "minimal Hodge theoretical dimension" AND "graph matroid" AND "communication complexity rank" - "algebraic geometry" AND "communication complexity" AND "Hodge dimensions"

Top relevant hits considered: - [s2:10.1109/ACCESS.2023.3301571] 3-D Beamforming Training Using Mutually Unbiased Bases for Cell Discovery in mm-Wave Systems - [s2:d0d995160a399bb372267bc86b38] THE RANKS OF $m \times n \times (mn)$ TENSORS *

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        # Generate a random graph using adjacency matrix representation
        adj_matrix = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    adj_matrix[i][j] = 1
                    adj_matrix[j][i] = 1
        return adj_matrix

    def matrix_multiplication(A, B):
        # Manually implement matrix multiplication
        result = [[sum(a * b for a, b in zip(row, col)) for col in zip(*B)] for row in A]
        return result

    def gaussian_elimination(matrix):
        n = len(matrix)
        augmented_matrix = [row[:] + [0] for row in matrix]
        for i in range(n):
            # Find the pivot
            max_row = i
            for j in range(i+1, n):
                if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
                    max_row = j
```

```

augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]

# Eliminate below the pivot
for j in range(i+1, n):
    factor = augmented_matrix[j][i] / augmented_matrix[i][i]
    for k in range(n + 1):
        augmented_matrix[j][k] -= factor * augmented_matrix[i][k]

# Back-substitute to find the solution
x = [0] * n
for i in range(n-1, -1, -1):
    x[i] = augmented_matrix[i][-1] / augmented_matrix[i][i]
    for j in range(i-1, -1, -1):
        augmented_matrix[j][-1] -= augmented_matrix[j][i] * x[i]
return x

def compute_hdim(G):
    # Compute the Hodge theoretical dimension (simplified version)
    n = len(G)
    identity = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    adj_matrix = G
    laplacian = matrix_multiplication(adj_matrix, adj_matrix)
    laplacian = [row[:] for row in laplacian]
    for i in range(n):
        laplacian[i][i] -= sum(row[i] for row in laplacian if row != laplacian[i])

    # Perform Gaussian elimination to find the rank
    rank = gaussian_elimination(laplacian)
    return n - rank.count(0)

def compute_ccr(G):
    # Compute the communication complexity rank (simplified version)
    n = len(G)
    identity = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    adj_matrix = G
    laplacian = matrix_multiplication(adj_matrix, adj_matrix)
    laplacian = [row[:] for row in laplacian]
    for i in range(n):
        laplacian[i][i] -= sum(row[i] for row in laplacian if row != laplacian[i])

    # Perform Gaussian elimination to find the rank
    rank = gaussian_elimination(laplacian)
    return n - rank.count(0)

results = []
for n in [5, 10, 15, 20, 30, 40]:
    hdim_values = []
    ccr_values = []
    for _ in range(5): # Test each size with 5 instances
        G = generate_random_graph(n)
        hdim = compute_hdim(G)
        ccr = compute_ccr(G)
        hdim_values.append(hdim)
        ccr_values.append(ccr)

    mean_hdim = sum(hdim_values) / len(hdim_values)

```

```

mean_ccr = sum(ccr_values) / len(ccr_values)
correlation_coefficient = (sum((h - mean_hdim) * (c - mean_ccr) for h, c in zip(hdim_values, ccr_values)) /
                           math.sqrt(sum((h - mean_hdim) ** 2 for h in hdim_values) *
                                       sum((c - mean_ccr) ** 2 for c in ccr_values)))

results.append({
    "n": n,
    "mean_hdim": mean_hdim,
    "mean_ccr": mean_ccr,
    "correlation_coefficient": correlation_coefficient
})

# Check if the correlation coefficient is significantly different from 0
p_value = 2 * (1 - math.erf(abs(results[0]["correlation_coefficient"]) / math.sqrt(2 * len(results))))

return {
    "metric_name": "Pearson Correlation Coefficient",
    "metric_value": results[0]["correlation_coefficient"],
    "instances_tested": 30,
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": abs(results[0]["correlation_coefficient"]) > 0.8 and p_value < 0.05,
    "counterexample": "" if abs(results[0]["correlation_coefficient"]) > 0.8 else "not_enough_instances"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='not_enough_instances' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ebe6613c.py", line 130, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ebe6613c.py", line 95, in run_trial
    hdim = compute_hdim(G)
            ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ebe6613c.py", line 72, in compute_hdim
    rank = gaussian_elimination(laplacian)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ebe6613c.py", line 49, in gaussian_elimination
    factor = augmented_matrix[j][i] / augmented_matrix[i][i]
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture’s support conditions. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13105
2	preregistration	ollama_remote	glm4:latest	0	0	9912
3	novelty	ollama_remote	glm4:latest	0	0	8723
4	novelty	ollama_remote	glm4:latest	0	0	14263
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21215
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11888
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10787
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18043
9	judge	ollama_remote	glm4:latest	0	0	18498

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 126434 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/af50c556b5f0.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/af50c556b5f0.jsonl for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/af50c556b5f0.tar.gz` (if generated)